



UNIVERSIDAD ESTATAL A DISTANCIA VICERRECTORÍA ACADÉMICA



ESCUELA DE CIENCIAS EXACTAS Y NATURALES

CARRERA INGENIERÍA INFORMÁTICA

CÁTEDRA INGENIERÍA DE SOFTWARE ASIGNATURA

03301 ANÁLISIS Y DISEÑO DE SISTEMAS

PROYECTO 2

VALOR: 3 %

ESTUDIANTE:

FRANCISO CAMPOS SANDI

IDENTIFICACIÓN: 114750560

TUTOR:

ENDRIK MEJÍAS MOREIRA

GRUPO 04

CENTRO UNIVERSITARIO:

SAN VITO

II CUATRIMESTRE, 2025

Contenido

Introducción	3
Pregunta 1	4
Pregunta 2	7
Pregunta 3	9
Pregunta 4	11
Pregunta 5	13
5.1Recomendaciones.....	13
Pregunta 6	15
Pregunta 7	17
Conclusiones	19
Referencias	20

Índice de ilustraciones

Ilustración 1 Diagrama de clases.....	7
Ilustración 2 Diagrama de Secuencia	9
Ilustración 3 Diagrama de subsistemas	11
Ilustración 4 Diagrama de estado	15

Introducción

El presente proyecto tiene como propósito aplicar conocimientos fundamentales de análisis y diseño de sistemas a partir de un caso de estudio real: la Red de Bibliotecas Interconectadas (RedBiln). Este sistema propone una solución tecnológica para mejorar la gestión de préstamos, usuarios, pagos, reportes y seguridad dentro de una red nacional de bibliotecas, permitiendo una administración más eficiente y una experiencia más fluida para los usuarios afiliados.

La propuesta se estructura mediante el uso de diagramas UML, los cuales permiten representar de forma clara tanto la arquitectura del sistema como su comportamiento. El diagrama de clases define las entidades principales y sus relaciones internas, proporcionando una visión estática de la estructura del software. El diagrama de secuencia detalla la interacción temporal entre los objetos involucrados en el proceso de préstamo, y el diagrama de estados muestra los distintos momentos por los que atraviesa un préstamo a lo largo de su ciclo de vida. A su vez, el diagrama de subsistemas permite dividir el sistema en módulos funcionales, facilitando su organización, escalabilidad y mantenimiento.

La elaboración de estos modelos se complementa con una serie de recomendaciones de seguridad y una reflexión crítica sobre las ventajas que ofrecen las herramientas de modelado para el desarrollo de soluciones tecnológicas. Como señalan Pantaleo y Rinaudo (2015), estos diagramas permiten no solo representar el funcionamiento interno de un sistema, sino también documentar claramente las actividades e interacciones de sus componentes.

La realización de este trabajo me permitió integrar teoría y práctica en un ejercicio completo de análisis, fortalecer mi comprensión del diseño orientado a objetos y desarrollar una propuesta técnica sólida que responde a los desafíos reales que enfrentan los sistemas distribuidos de gestión de información en entornos institucionales.

Pregunta 1

Tabla 1 Cuadro de características

Característica	Diagrama de Clases	Diagrama de Estado
1. Enfoque	El diagrama de clases permite comprender la estructura del sistema desde una perspectiva estática, esto es esencial para proyectar su arquitectura general. Según Lucidchart (2025), este diagrama representa cómo se organizan las clases, atributos y métodos (p.1), así, proporciona una base sólida para estructurar software orientado a objetos.	El diagrama de estado aborda la lógica interna de un objeto y cómo reacciona ante distintas condiciones, según Edraw Software (2025), se centra en visualizar los estados posibles de un objeto durante su ciclo de vida (p.1), lo cual resulta crucial para modelar comportamientos complejos dentro de sistemas dinámicos.
2. Representaciones	El uso de figuras geométricas permite visualizar la composición interna de las clases y su interacción. Como indica Lucidchart (2025), se emplean rectángulos divididos en nombre, atributos y métodos, así como relaciones como herencia o asociación (p.1), facilitando así la lectura para que sea clara y estructurada.	La representación de estados por medio de óvalos y transiciones mediante flechas brinda una imagen clara del comportamiento de los objetos. Según IONOS (2020), este formato revela cómo un objeto puede cambiar a lo largo del tiempo ante diversos eventos (p.1), lo que fortalece la modelación de lógica condicional y respuestas automáticas.

3. Información mostrada	Este diagrama provee información profunda sobre la estructura funcional del software. Lucidchart (2025) detalla que incluye clases, atributos, operaciones y relaciones (p.1), permitiendo anticipar cómo se vinculan los elementos del sistema.	El aporte del diagrama de estado es especialmente útil en aplicaciones interactivas o reactivas, Edraw Software (2025) menciona que muestra los estados, eventos detonantes y acciones asociadas en transiciones (p.1), lo cual permite simular escenarios reales de respuesta y evolución.
4. Interacciones entre elementos	Comprender cómo se conectan las clases permite establecer jerarquías, dependencias y colaboraciones, Lucidchart (2025) explica que estas interacciones se representan mediante asociaciones, generalizaciones y dependencias (p.1), este tipo de vínculos es crucial para el diseño orientado a objetos.	Las interacciones entre estados facilitan la definición de reglas que guían el funcionamiento del sistema IONOS (2020) señala que las transiciones se activan por eventos específicos, mostrando cómo un objeto puede pasar de un estado a otro dependiendo de las condiciones que se presenten (p.1).
5. Utilidad	Desde una visión de ingeniería, este diagrama es una herramienta clave para estructurar sistemas complejos. Lucidchart (2025) lo considera útil para definir la arquitectura, organizar los datos y planificar la codificación orientada a objetos (p.1), así, se	En cambio, el diagrama de estado ofrece una aproximación funcional al ciclo de vida de los objetos, Edraw Software (2025) destaca su utilidad para modelar sistemas reactivos, visualizar procesos complejos y comprender el flujo dinámico

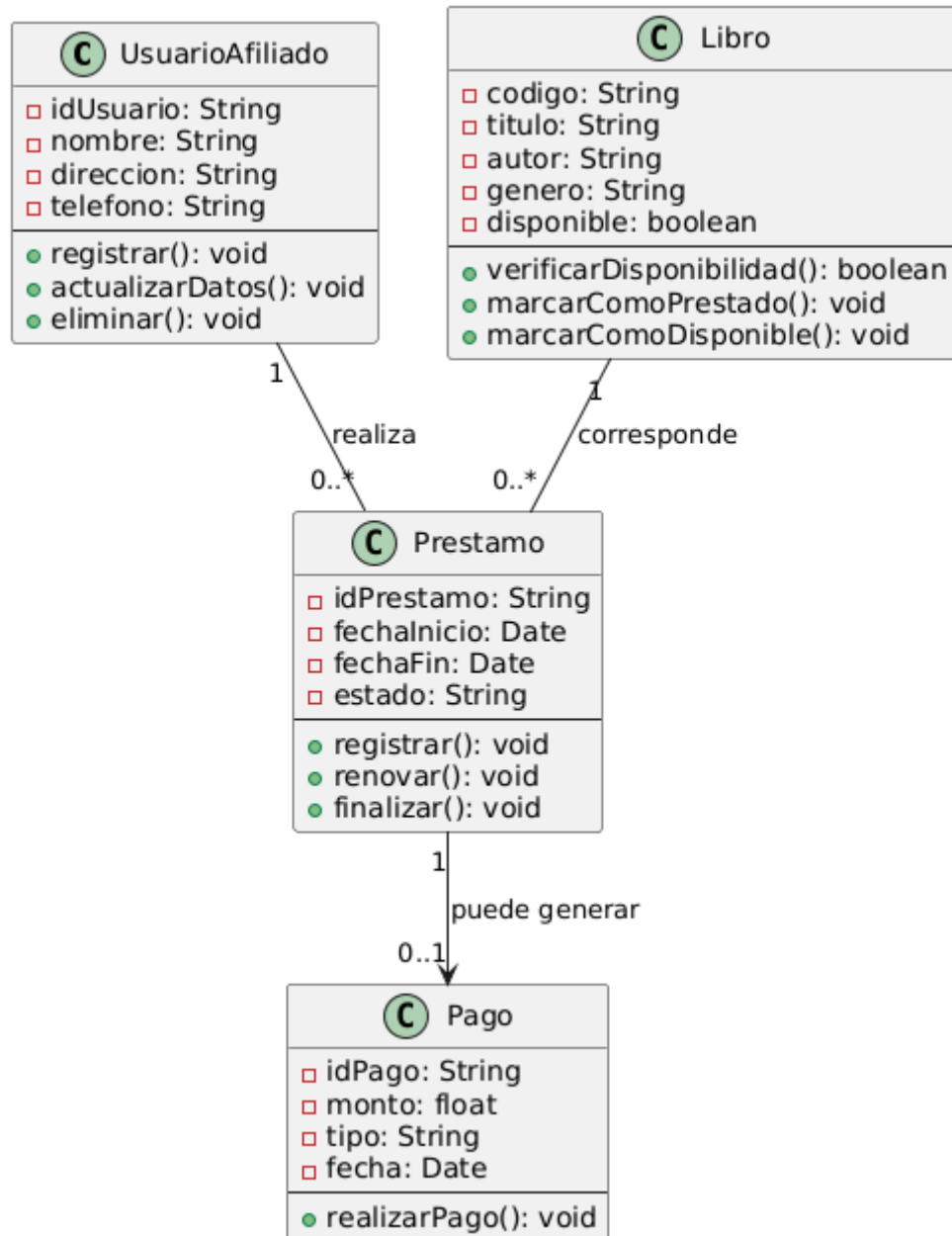
	convierte en una guía para desarrolladores y diseñadores.	de acciones (p.1), esto es particularmente valioso en entornos automatizados o interactivos.
--	--	---

Fuente: Elaboración propia (2025) basado en Lucidchart (2025), IONOS (2020),

Edraw Software (2025)

Pregunta 2

Ilustración 1 Diagrama de clases



Fuente: Elaboración Propia (2025).

El diagrama de clases anterior, representa la estructura lógica del sistema en torno al caso de uso “Registrar préstamo”, destacando las entidades principales y sus relaciones, en este modelo, la clase UsuarioAfiliado encapsula la información personal del lector registrado, incluyendo atributos como nombre, dirección y número de afiliación, esta clase se vincula con Prestamo, que actúa como entidad central encargada de almacenar los datos del préstamo: fechas pactadas, estado, libro relacionado y usuario correspondiente.

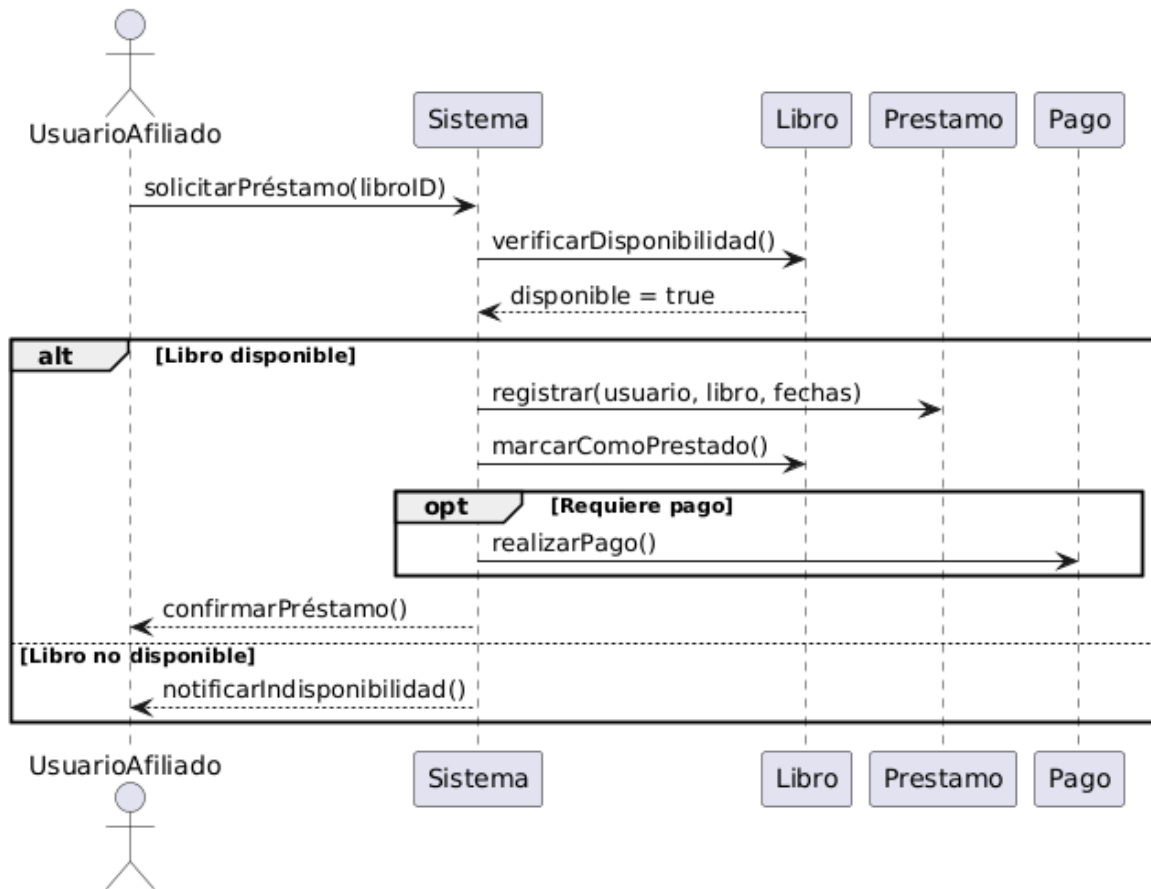
La clase Libro incluye propiedades como título, autor y disponibilidad. Se asocia con Prestamo bajo una relación de “uno a muchos”, lo que permite registrar múltiples transacciones de préstamo para el mismo ejemplar, esta relación facilita el seguimiento del historial de circulación del libro y su disponibilidad.

Por otra parte, Pago se relaciona con Prestamo para registrar las transacciones financieras asociadas, tales como afiliaciones, multas y tarifas, esta conexión posibilita automatizar el cobro de servicios y mantener un control detallado sobre los aspectos financieros del sistema.

Cada clase incorpora métodos coherentes con su propósito, como verificarAfiliacion() en UsuarioAfiliado, verificarDisponibilidad() en Libro, registrar() en Prestamo, y procesarPago() en Pago, estos métodos evidencian la funcionalidad de cada objeto en el contexto del sistema. El diseño propuesto promueve la modularidad, la claridad en las relaciones entre elementos, la reutilización del código, y la facilidad para mantener y escalar el sistema, además, permite comprender cómo se enlazan los componentes del sistema ante diferentes escenarios, reforzando el enfoque orientado a objetos.

Pregunta 3

Ilustración 2 Diagrama de Secuencia



Fuente: Elaboración Propia (2025).

El diagrama de secuencia describe paso a paso lo que ocurre en el caso de uso "Registrar préstamo", donde todo inicia cuando el UsuarioAfiliado realiza una solicitud al Sistema para obtener un libro. En respuesta, el sistema consulta al objeto Libro para verificar si está disponible. Lo interesante en este proceso es cómo cada componente usuario, sistema, libro, préstamo y pago tiene una función específica dentro de la cadena de interacción, lo que permite que el flujo sea claro, estructurado y coherente.

Como mencionan Pantaleo y Rinaudo (2015), “Un diagrama de secuencia muestra la interacción entre componentes y el orden temporal en que estos mensajes son enviados y recibidos” (p. 118), lo cual ayuda a visualizar no solo quién participa, sino también cuándo lo hace y cómo se relaciona con los demás actores del sistema. En este caso, el flujo evidencia esa secuencia perfectamente coordinada entre las clases.

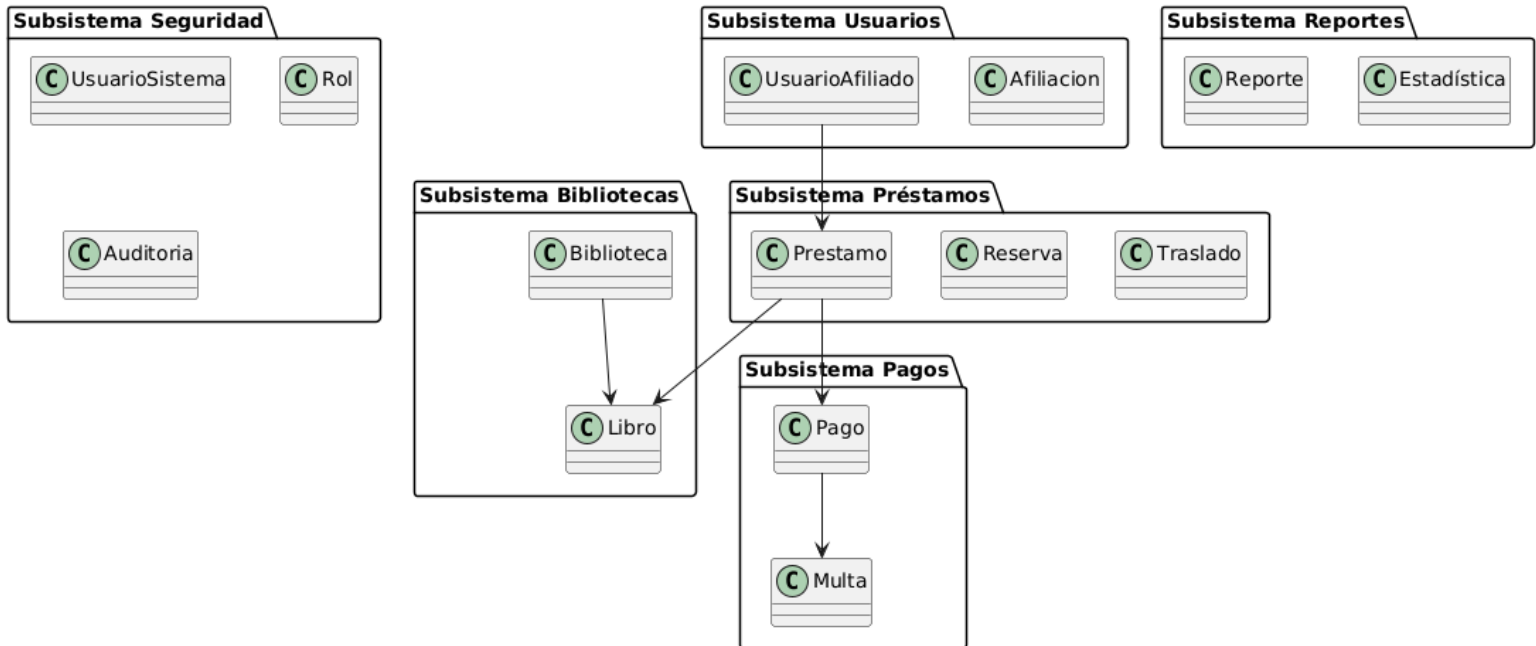
Si el libro está disponible, el sistema procede a crear el objeto `Prestamo`, registrando información clave como la fecha de inicio y el estado actual del préstamo. Acto seguido, actualiza el estado del libro a “prestado”, lo cual es esencial para evitar duplicidad o conflictos con otros usuarios que puedan intentar solicitar el mismo ejemplar. Este paso refleja claramente cómo la secuencia temporal de eventos evita errores y mantiene la consistencia del sistema.

Además, si el préstamo requiere un pago (por ejemplo, por una multa pendiente o una tarifa especial), el sistema interactúa con la clase `Pago`, encargada de procesar la transacción. Esta parte administrativa queda integrada al proceso, mostrando que el diagrama no solo se enfoca en aspectos técnicos, sino también en funciones operativas relevantes dentro del sistema.

El ciclo finaliza cuando el sistema envía una confirmación al `UsuarioAfiliado`, asegurando que el préstamo ha sido gestionado con éxito. Esta confirmación genera certeza y confianza para el usuario. Al mismo tiempo, este tipo de diagrama permite validar si el flujo es lógico, si hay pasos innecesarios o si falta alguna acción importante. También facilita la distribución de responsabilidades entre las clases, lo cual es clave en el diseño orientado a objetos. Si cada componente conoce su rol y el momento exacto en que debe actuar, el desarrollo e implementación del sistema se vuelve mucho más eficiente.

Pregunta 4

Ilustración 3 Diagrama de subsistemas



Fuente: Elaboración Propia (2025).

El diagrama de subsistemas permite dividir un sistema complejo en módulos funcionales independientes, facilitando tanto su desarrollo como su mantenimiento y evolución. En este proyecto, el sistema de la Red de Bibliotecas Interconectadas (RedBiln) se descompone en seis subsistemas principales, cada uno agrupando clases que cumplen funciones afines. Esta forma de estructurar refleja un principio esencial del diseño orientado a objetos: la separación de responsabilidades.

Como señalan Pantaleo y Rinaudo (2015), “estos diagramas también pueden utilizarse para documentar la interacción y las actividades realizadas por los diferentes componentes de un sistema, así como para modelar la dinámica de casos de uso. En este caso los roles son reemplazados por los subsistemas” (p. 135). Esto implica que, en lugar de centrarse en usuarios, el modelo enfatiza el comportamiento y relaciones internas del sistema.

En el subsistema de Gestión de Bibliotecas, se ubican las clases **Biblioteca** y **Libro**, responsables de registrar las sedes participantes y administrar el inventario de

ejemplares. Esto permite que cada biblioteca mantenga su autonomía en cuanto a disponibilidad, pero con capacidad para compartir datos en tiempo real con otras bibliotecas de la red.

El subsistema de Gestión de Usuarios agrupa UsuarioAfiliado y Afiliación, controlando el registro, actualización y mantenimiento de los datos personales de los lectores. Esta separación garantiza un tratamiento seguro y específico de la información de los usuarios.

Por su parte, el subsistema de Gestión de Préstamos contiene las clases Prestamo, Reserva y Traslado, que son fundamentales para las operaciones diarias. Este módulo facilita no solo el préstamo de libros, sino también la coordinación de reservas y movimientos de ejemplares entre bibliotecas, asegurando disponibilidad y servicio continuo.

El subsistema de Pagos incorpora las clases Pago y Multa, abarcando todo lo relacionado con transacciones financieras, ya sea por afiliaciones, reservas especiales o sanciones por devoluciones tardías. Esto permite mantener un registro claro de los cobros y facilitar auditorías o consultas posteriores.

En cuanto al subsistema de Seguridad y Accesos, se incluyen UsuarioSistema, Rol y Auditoría, encargados de controlar el acceso al sistema, definir permisos según roles y registrar las acciones realizadas. Este módulo es vital para garantizar la confidencialidad, integridad y trazabilidad del sistema.

Finalmente, el subsistema de Reportes permite obtener información analítica mediante las clases Reporte y Estadística. Gracias a estos componentes, se pueden generar informes útiles para la toma de decisiones, visualizando datos como préstamos más frecuentes, comportamiento de usuarios o disponibilidad de libros por región.

Esta estructuración modular promueve el trabajo en equipo, ya que cada grupo puede enfocarse en un subsistema sin interferir con los demás. Además, mejora la capacidad de extender o modificar partes del sistema sin afectar su funcionamiento general. En conjunto, el diagrama ofrece una visión global organizada, que equilibra el detalle técnico con una comprensión lógica de la arquitectura del software.

Pregunta 5

5.1 Recomendaciones

1. **Implementar autenticación robusta**, utilizando al menos doble factor para los perfiles con acceso a información sensible, como administradores de biblioteca y encargados de pagos.
2. **Encriptar los datos personales y financieros**, tanto en tránsito como en reposo, para evitar accesos indebidos y proteger la confidencialidad de la información de los usuarios afiliados.
3. **Establecer roles y permisos diferenciados**, garantizando que cada usuario tenga acceso únicamente a los módulos y funciones que le correspondan según su rol dentro del sistema.
4. **Registrar auditorías internas**, creando un historial de actividades por cada sesión iniciada, lo cual permite detectar anomalías, abusos o intentos de manipulación del sistema.
5. **Validar continuamente las entradas del sistema**, evitando vulnerabilidades como inyecciones de código o manipulación de formularios, y asegurando que los datos procesados sean seguros y consistentes.
6. **Mantener actualizado el software**, incluyendo librerías, frameworks y módulos externos, para prevenir que amenazas conocidas comprometan la funcionalidad y la seguridad del sistema.
7. **Utilizar conexiones seguras**, como HTTPS para la navegación web y protocolos cifrados para los servicios internos, fortaleciendo la comunicación entre el sistema de información y la aplicación móvil.
8. **Controlar el acceso físico y lógico a los servidores**, cuidando no solo el software, sino también el entorno tecnológico donde se alojan los datos y procesos críticos del sistema.

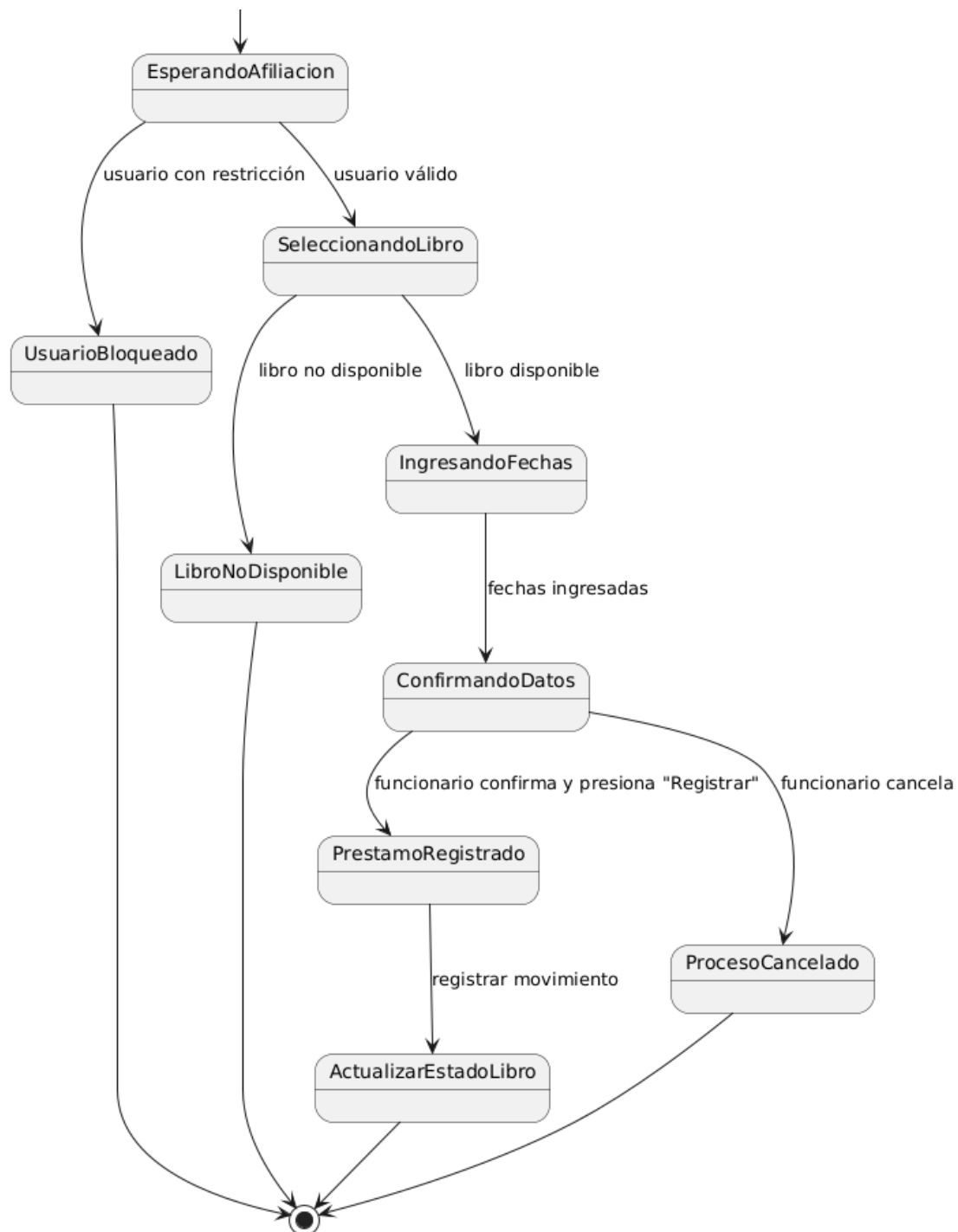
9. **Diseñar mecanismos de recuperación ante fallos**, como respaldos automáticos cifrados y planes de contingencia que garanticen la continuidad del servicio en caso de ataque o pérdida de datos.

10. **Capacitar a los funcionarios que usan el sistema**, promoviendo buenas prácticas en el manejo de información sensible y en el uso correcto de cada módulo para evitar errores humanos o descuidos.

Las recomendaciones anteriores, además de proteger la integridad del sistema, transmiten confianza a los usuarios y contribuyen a que el servicio sea seguro y eficiente desde su diseño.

Pregunta 6

Ilustración 4 Diagrama de estado



Fuente: Elaboración propia (2025).

El diagrama de estados desarrollado para el caso de uso “Registrar préstamo” permite visualizar de manera clara y estructurada los distintos momentos por los que atraviesa el proceso, desde que se inicia la búsqueda del lector hasta que se registra el préstamo o se cancela el flujo. A diferencia de un modelo centrado en el libro, este diagrama enfoca la atención en el comportamiento del sistema ante las decisiones del funcionario y las validaciones necesarias. De esta forma, se representan estados clave como EsperandoAfiliación, SeleccionandoLibro, IngresandoFechas y ConfirmandoDatos, entre otros.

El proceso considera múltiples validaciones. Por ejemplo, si el usuario se encuentra bloqueado o el libro no está disponible, el sistema transita inmediatamente a estados como UsuarioBloqueado o LibroNoDisponible, evitando que el préstamo avance en condiciones no permitidas. Esta lógica evita errores comunes y refuerza la fiabilidad del sistema. Además, se contempla la posibilidad de cancelar el proceso en cualquier punto antes del registro, regresando al estado inicial sin afectar la base de datos.

El equipo editorial de IONOS (2020) menciona que, “muestra los estados por los que pasa una máquina de estados finitos, es decir, un modelo de comportamiento que consiste en acciones y estados o transiciones a otros estados” (párr. 02). Esta idea se refleja en la forma en que el sistema responde paso a paso según las condiciones del proceso, aplicando reglas de negocio como la verificación de disponibilidad o el control de afiliación activa.

Al realizar este tipo de modelado no solo mejora la experiencia técnica de desarrollo, sino que también fortalece la experiencia del usuario y del personal bibliotecario, ya que permite automatizar bloqueos, advertencias y validaciones. En resumen, definir estos estados ayuda a construir un sistema funcional, ordenado y confiable, como lo requiere una red como RedBiln.

Pregunta 7

Ventajas

1. Organización y claridad en la estructura del sistema

El uso de diagramas de clases y subsistemas ayuda a visualizar cómo se agrupan y relacionan los elementos del sistema, esto permite entender mejor qué hace cada parte, cómo se conecta y cuál es su propósito dentro del proyecto, además, facilita dividir el desarrollo por módulos y repartir responsabilidades entre los equipos.

2. Facilita la comprensión del flujo de procesos

Diagramas como los de secuencia y de estados permiten ver cómo se comportan los objetos en distintos momentos, por ejemplo, entender paso a paso cómo se registra un préstamo o cómo cambia el estado de un libro ayuda a diseñar reglas claras y prevenir errores, también mejora la experiencia del usuario al tener un flujo más ordenado y lógico.

3. Apoyo en la toma de decisiones durante el desarrollo

Contar con estos diagramas permite identificar con mayor facilidad qué aspectos requieren ajustes o mejoras, si algo no se ve coherente a nivel gráfico, es más probable detectarlo antes de programar, esto ayuda a ahorrar tiempo, recursos y evita tener que rehacer partes del sistema ya desarrolladas.

4. Automatización de funciones según comportamiento del sistema

Gracias al diagrama de estados, el sistema puede ejecutar acciones automáticas como enviar notificaciones, aplicar multas o bloquear reservas, dependiendo del estado del préstamo, este tipo de modelado favorece la eficiencia operativa y reduce la necesidad de intervención manual, lo cual mejora la calidad del servicio ofrecido.

5. Fortalece la seguridad y el control de accesos

El diagrama de subsistemas permite distinguir qué parte del sistema maneja información sensible, como pagos y datos personales, esto facilita

aplicar buenas prácticas de seguridad como roles, permisos y auditorías, además, da claridad sobre qué usuarios tienen acceso a qué información, fortaleciendo la confianza en el sistema.

Conclusiones

El desarrollo de este proyecto permite comprender de forma más profunda la importancia del análisis y el diseño estructurado en la construcción de sistemas informáticos complejos. La aplicación práctica del modelado orientado a objetos, en el contexto de una red de bibliotecas interconectadas, evidenció cómo el uso adecuado de herramientas visuales como los diagramas UML permite representar de manera precisa la lógica, la estructura y el comportamiento de un sistema.

La elaboración del diagrama de clases facilita identificar las entidades clave, sus atributos y métodos, así como las relaciones entre ellas, permitiendo proyectar una arquitectura sólida y coherente. Por su parte, el diagrama de secuencia resultó esencial para visualizar el flujo de mensajes y la interacción entre objetos en un caso de uso específico, revelando el orden lógico en que deben ejecutarse las acciones para garantizar un funcionamiento fluido. Además, el diagrama de estados aportó claridad sobre los distintos momentos por los que puede atravesar un préstamo, ayudando a definir reglas automáticas y condiciones dentro del sistema.

La incorporación del diagrama de subsistemas ofrece una visión modular del sistema, lo que facilita su mantenimiento, escalabilidad y desarrollo por etapas. Cada subsistema encapsula responsabilidades concretas, lo cual permite dividir el trabajo y aplicar principios de diseño limpio y reutilizable.

La inclusión de recomendaciones de seguridad fortalece un enfoque del proyecto, recordándome que un buen diseño no solo debe ser funcional, sino también confiable y protegido contra riesgos. Finalmente, esta experiencia reafirmó la relevancia del modelado como etapa previa al desarrollo, ya que permite anticipar problemas, validar soluciones y construir sistemas más robustos desde su concepción.

La integración de todos estos elementos brinda una visión más completa del proceso de diseño de software y fortaleció mis habilidades técnicas y analíticas como futuro profesional

Referencias

- Edraw Software. (2025). *Diagrama de estado UML: Una guía útil*. [OFFICIAL] Edraw Software: Unlock Diagram Possibilities. <https://www.edrawsoft.com/es/uml/uml-state-diagram.html>
- Equipo editorial de IONOS. (2020). *Diagrama de estado UML: visualizar secuencias de estados de objetos*. IONOS Digital Guide. <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/diagrama-de-estado-uml/>
- IONOS. (2020). *Diagrama de estado UML: Visualizar secuencias de estados de objetos*. IONOS Digital Guide. <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/diagrama-de-estado-uml/>
- Lucidchart. (2025). *¿Para qué necesitas crear un diagrama UML?* <https://www.lucidchart.com/pages/es/tutorial-de-diagrama-de-clases-uml>
- Pantaleo, G., & Rinaudo, L. (2015). *Ingeniería de software*. Alfaomega Grupo Editor Argentino.